

Introduction to Stack and Queues

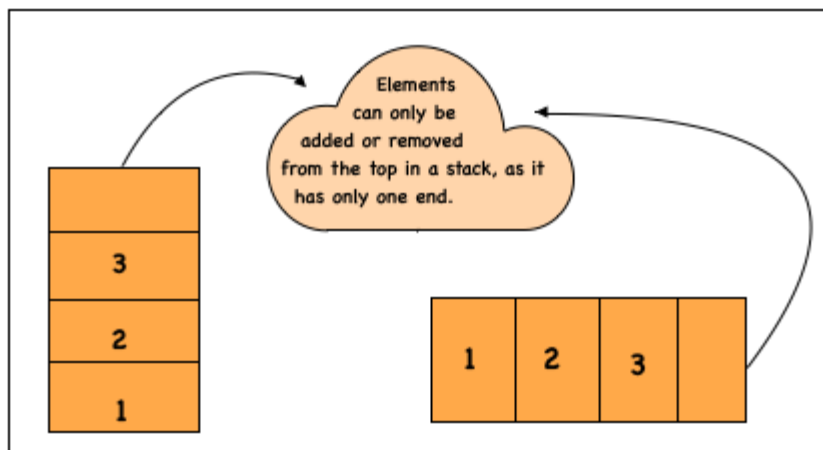
What is a Stack?

Stack follows the **LIFO (Last-In-First-Out)** principle. The last element added is the first one to come out.

Stack Operations

- **Push:** Add an element to the top
- **Pop:** Remove the top element
- **Peek/Top:** View the top-most element

Visualisation:



Example Flow

- `stack.push(1)` → [1]
- `stack.push(3)` → [1, 3]
- `stack.pop()` → returns 3 → [1]
- `stack.peak()` → returns 1

Examples

- Stacks of Books
- Undo/Redo feature

- Recursion (Function Call Stack)
- Browsing History

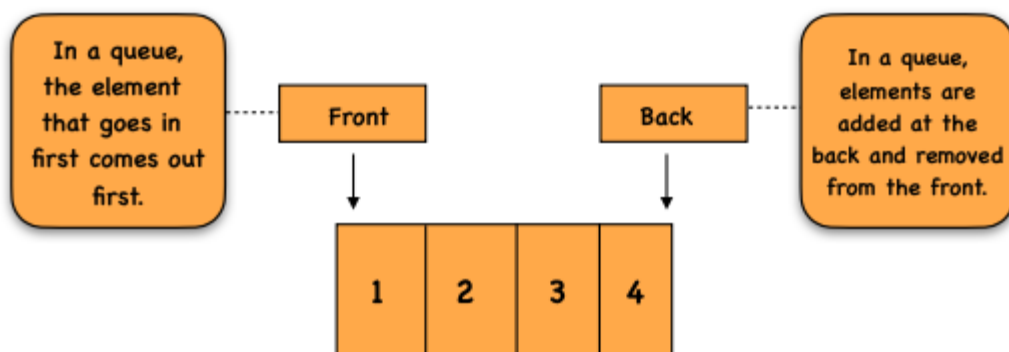
What is a Queue?

Queue follows the **FIFO (First-In-First-Out)** principle. The first element added is the first to be removed.

Queue Operations

- **Enqueue:** Add an element to the end
- **Dequeue:** Remove element from the front
- **Peek/Front:** View the front-most element

Visualisation:



Examples

- Ticket Counter
- OS Task Scheduling
- Printers
- Queues (in general)

Why Use Stack and Queue ?

- Organize data logically (based on problem's need).
- Order of Operations Matter
- Optimize time & memory for specific scenarios

Use Cases

- Recursion (stack)
- Level Order Traversal (queue)
- BFS (queue)
- DFS (stack)

Language Implementation

JavaScript

```
let stack = [];  
stack.push(10);  
stack.push(7);  
stack.pop(); // returns 7  
stack.pop(); // returns 10
```

Java

```
Stack stack = new Stack<>();  
stack.push(10);  
stack.pop();  
stack.peek();
```

C++

```
#include <stack>  
std::stack s;  
s.push(10);  
s.pop();
```

```
s.top();
```

Python

```
stack = []
```

```
stack.append(10)
```

```
stack.pop()
```

```
top = stack[-1] # peek
```

Note: Find out more about stacks in your preferred programming language.

Stack vs Queue

 Stack

- Last-In-First-Out
- Access only from top
- Used in DFS, Recursion

 Queue

- First-In-First-Out
- Access from front & rear
- Used in BFS, Scheduling

comparison with other Data-Structures

Features	Stack	Queue	Array	Linked List	HashMap
Order	LIFO	FIFO	Indexed	Sequential	Key-based
Random Access	✗	✗	✓	✗	✓
Inser/Delete	Top Only	Front/back	Anywhere	Anywhere	By Key
Time Complexity	O(1)	O(1)	Varies	O(1) insert	O(1)
Use Case	Backtracking	Scheduling	General Purpose	Dynamic Size	Lookup

Playing with Stacks & Queues

Common Operations → Stack

- **Push:** Add an element to the top of the stack
- **Pop:** Remove the top-most element from the stack
- **Peek/Top:** View the top-most element without removing it

Stack in JavaScript

```
let stack = []; // Simply, Use Array as Stack.
```

```
stack.push(1); // Add inside stack
```

```
stack.push(2);
```

```
stack.push(3);
```

```
console.log(stack); // [1, 2, 3]
```

```
stack.pop(); // A stack is LIFO (Last In, First Out), so the element that goes in last will come out first. In this case, 3 will be removed.
```

```
stack.pop();
```

```
console.log(stack); // [1]
```

```
stack.push(7); // [1, 7]
```

```
// For Top element
```

```
let top = stack[stack.length - 1];
```

```
console.log(top); // 7
```

```
// Invalid: stack[3] - violates stack discipline
```

```
// It works fine because it's treated as an array, but this is invalid in the context of a stack.
```

```
// Always use push, pop, and peek for proper stack usage.
```

Queue:

A **Queue** is a **FIFO (First-In-First-Out)** data structure where the first element added is the first to be removed.

Common Operations → Queue

- **Enqueue:** Add an element to the end of the queue
- **Dequeue:** Remove the element from the front of the queue
- **Peek/Front:** View the front element without removing it

Queue in JavaScript

```
let q = [];
```

```
// Enqueue
```

```
q.push(1);
```

```
q.push(2);  
q.push(3);  
console.log(q); // [1, 2, 3]
```

```
// Dequeue: Remove from front of the Queue
```

```
q.shift(); // Removes 1
```

```
// Peek: Get first element of queue.
```

```
let front = q[0];
```

```
console.log(front); // 2
```

```
// Invalid Syntax:
```

```
q.pop(); // This works because it's an array method, but  
it removes from the end and breaks queue logic.
```

```
// Queue should only remove from the front.
```

Implement Stack using Two Queues

Problem Statement:

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (`push`, `top`, `pop`, and `empty`).

Implement the `MyStack` class:

- `void push(int x)` Pushes element x to the top of the stack.
- `int pop()` Removes the element on the top of the stack and returns it..
- `int top()` Returns the element on the top of the stack.
- `boolean empty()` Returns true if the stack is empty, `false` otherwise.

Notes

- You must use only standard operations of a queue, which means that only **push to back**, **peek/pop from front**, **size** and **is empty** operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Examples

Example 1:

Input: ["MyStack", "push", "push", "top", "pop", "empty"] [], [1], [2], [], [], []

Output: [null, null, null, 2, 2, false]

Explanation MyStack myStack = new MyStack(); myStack.push(1); myStack.push(2); myStack.top(); // return 2 myStack.pop(); // return 2 myStack.empty(); // return False

Constraints:

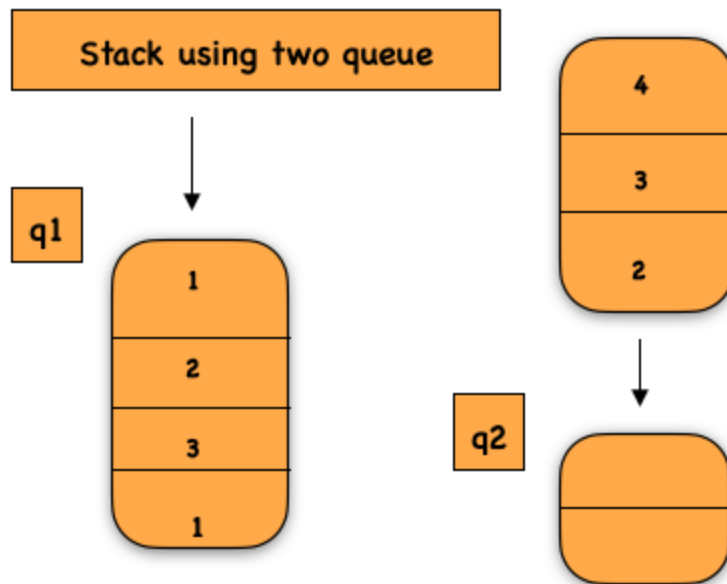
- $1 \leq x \leq 9$
- At most **100** calls will be made to **push**, **pop**, **top**, and **empty**.
- All the calls to **pop** and **top** are valid.

Follow-up: Can you implement the stack using only one queue?

Approach

- Initialize Two Queues.
- Just add the element to q1. (q1.push(x))
- Remove the last pushed element (i.e., rear of q1).
- Similar to pop(), but instead of removing the last element.
- Check if q1 is empty.

Visualisation:



Time Complexity:

- **push** $\rightarrow O(1)$
- **pop/top** $\rightarrow O(n)$
- **empty** $\rightarrow O(1)$

Space Complexity:

- **Space Complexity** = $O(1)$
- In-place modifications with constant extra space.

Dry Run

Input:

Operations: push(1) push(2) push(3) top() pop() top() empty()

State Transitions:

After push(1): q1 = [1] q2 = []
After push(2):
q1 = [1, 2]

```

q2 = []
After push(3):
q1 = [1, 2, 3]
q2 = []
top():
Move q1 → q2 (except last):
q2 = [1, 2]
q1 = [3]
Peek front = 3, push it to q2
q2 = [1, 2, 3]
Swap q1 and q2
q1 = [1, 2, 3]
q2 = []
→ Returns: 3
pop():
Move q1 → q2 (except last):
q2 = [1, 2]
q1 = [3]
Remove last from q1: 3
Swap q1 and q2
q1 = [1, 2]
q2 = []
→ Returns: 3
top():
Move q1 → q2 (except last):
q2 = [1]
q1 = [2]
Peek front = 2, push it to q2
q2 = [1, 2]
Swap q1 and q2
q1 = [1, 2]
q2 = []
→ Returns: 2
empty():
q1 = [1, 2] → Not empty
→ Returns: false

```

Final State:	q1 = [1, 2],	Stack (top to bottom): [2, 1]
---------------------	--------------	-------------------------------

```
var MyStack = function() {
  this.q1 = [];
  this.q2 = [];
};

MyStack.prototype.push = function(x) {
  this.q1.push(x);
};

MyStack.prototype.pop = function() {
  let n = this.q1.length;
  for (let i = 0; i < n - 1; i++) {
    this.q2.push(this.q1.shift());
  }
  let ans = this.q1.shift();
  let temp = this.q1;
  this.q1 = this.q2;
  this.q2 = temp;
  return ans;
};

MyStack.prototype.top = function() {
  let n = this.q1.length;
  for (let i = 0; i < n - 1; i++) {
    this.q2.push(this.q1.shift());
  }
  let front = this.q1[0];
  this.q2.push(this.q1.shift());
  let temp = this.q1;
  this.q1 = this.q2;
  this.q2 = temp;
  return front;
};

MyStack.prototype.empty = function() {
  return this.q1.length === 0;
};
```

Implement Stack Using One Queue

Problem Statement:

Implement a last-in-first-out (LIFO) stack using only two queues. The implemented stack should support all the functions of a normal stack (`push`, `top`, `pop`, and `empty`).

Implement the `MyStack` class:

- `void push(int x)` Pushes element x to the top of the stack.
- `int pop()` Removes the element on the top of the stack and returns it..
- `int top()` Returns the element on the top of the stack.
- `boolean empty()` Returns true if the stack is empty, `false` otherwise.

Notes

- You must use only standard operations of a queue, which means that only `push to back`, `peek/pop from front`, `size` and `is empty` operations are valid.
- Depending on your language, the queue may not be supported natively. You may simulate a queue using a list or deque (double-ended queue) as long as you use only a queue's standard operations.

Examples

Example 1:

Input: `["MyStack", "push", "push", "top", "pop", "empty"]` `[[], [1], [2], [], [], []]`

Output: `[null, null, null, 2, 2, false]`

Explanation `MyStack myStack = new MyStack(); myStack.push(1); myStack.push(2); myStack.top(); // return 2 myStack.pop(); // return 2 myStack.empty(); // return False`

Constraints:

- $1 \leq x \leq 9$

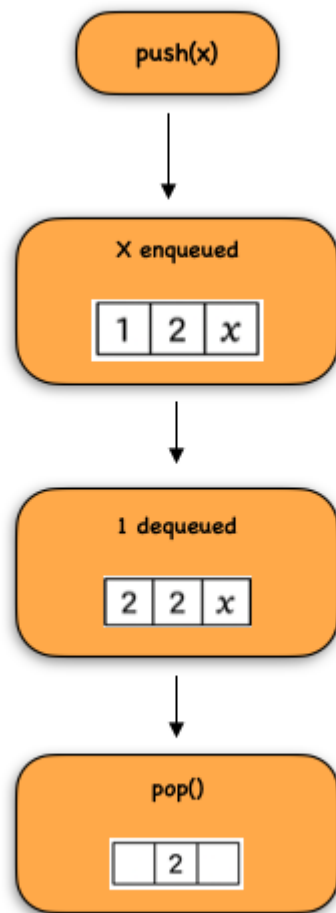
- At most **100** calls will be made to **push**, **pop**, **top**, and **empty**.
- All the calls to **pop** and **top** are valid.

Follow-up: Can you implement the stack using only one queue?

Approach: Implement Stack Using One Queue

- We simulate LIFO (stack) using only FIFO (queue) operations.
- We maintain a single queue (q).
- Remove the last pushed element (i.e., rear of q1).
 - Insert it at the back (q.push(x)).
 - Rotate the queue to bring the new element to the front by dequeuing and enqueueing all previous elements.

Visualisation:



Time Complexity:

- **push** → $O(n)$

pop → $O(1)$

top → $O(1)$

empty → $O(1)$

Space Complexity:

- **Space Complexity** = $O(n)$

- Where n is the total number of elements in the stack.

Dry Run

Input:

Operations:

```
push(1)
push(2)
push(3)
top()
pop()
top()
empty()
```

State Transitions:

After push(1):

q = [1]

After push(2):

q = [1, 2]

After push(3):

q = [1, 2, 3]

top():

Loop to rotate first n-1 = 2 elements:

q.push(q.shift()) → q = [2, 3, 1]

q.push(q.shift()) → q = [3, 1, 2]

Peek front = 3

Push front back → q = [1, 2, 3]

→ Returns: 3

pop():

Loop to rotate first n-1 = 2 elements:

q.push(q.shift()) → q = [2, 3, 1]

q.push(q.shift()) → q = [3, 1, 2]

Remove front = 3 → q = [1, 2]

→ Returns: 3

```
top():
    Loop to rotate first n-1 = 1 element:
        q.push(q.shift()) → q = [2, 1]
    Peek front = 2
    Push front back → q = [1, 2]
    → Returns: 2
```

```
empty():
    q = [1, 2] → Not empty
    → Returns: false
```

Final State:	q = [1, 2],	Stack (top to bottom): [2, 1]
---------------------	-------------	-------------------------------

Implement Queue using Stacks

Problem Statement:

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (`push`, `peek`, `pop`, and `empty`).

Implement the `MyQueue` class:

- `void push(int x)` Pushes element x to the back of the queue.
- `int pop()` Removes the element from the front of the queue and returns it.
- `int peek()` Returns the element at the front of the queue.
- `boolean empty()` Returns `true` if the queue is empty, `false` otherwise.

Notes

- You must use only standard operations of a stack, which means only `push to top`, `peek/pop from top`, `size` and `is empty` operations are valid.

- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Examples

Example 1:

Input: ["MyQueue", "push", "push", "top", "pop", "empty"] [[], [1], [2], [], [], []]

Output: [null, null, null, 1, 1, false]

Explanation

```
MyQueue myQueue = new MyQueue();
myQueue.push(1);    // queue is: [1]
myQueue.push(2);    // queue is: [1, 2]
                    (leftmost is front of the queue)
myQueue.peek();     // return 1
myQueue.pop();      // return 1, queue is [2]
myQueue.empty();    // return false
```

Constraints:

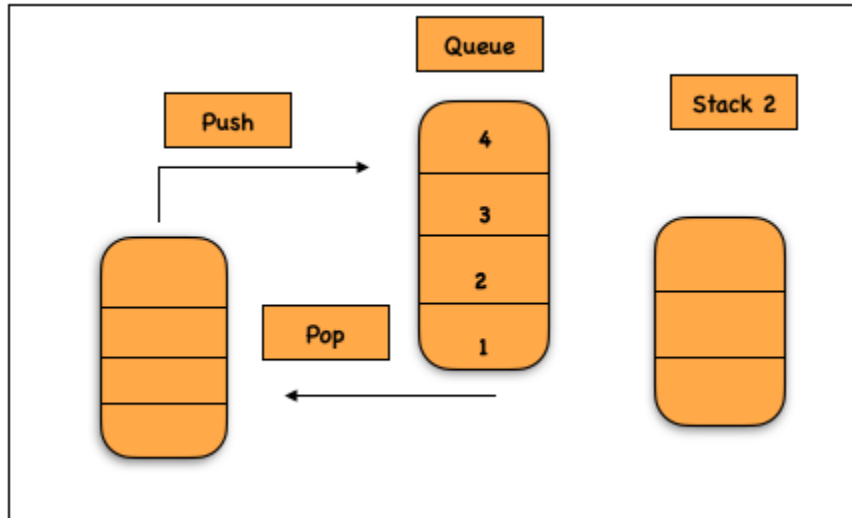
- $1 \leq x \leq 9$
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is amortized $O(1)$ time complexity? In other words, performing n operations will take overall $O(n)$ time even if one of those operations may take longer.

Approach: Using Two Stacks (s1 & s2)

- Use two stacks, s1 and s2.
- For Push, push onto s1.
- For pop, transfer elements from s1 to s2 if s2 is empty, then pop from s2.
- For peek, transfer elements from s1 to s2 if s2 is empty, then return top of s2.

Visualisation:



Time Complexity:

- `push` → $O(1)$

`pop` → $O(n)$

`peek` → $O(n)$

Space Complexity:

- **Space Complexity = $O(n)$**

Dry Run

Input:

Operations: `push(1)` `push(2)` `push(3)` `peek()` `pop()` `peek()`
`empty()`

State Transitions:

After `push(1)`: `s1 = [1]` `s2 = []`
After `push(2)`:
`s1 = [1, 2]`
`s2 = []`
After `push(3)`:

```
s1 = [1, 2, 3]
s2 = []
peek():
→ s2 is empty → move all from s1 to s2
s1.pop() = 3 → s2.push(3)
s1.pop() = 2 → s2.push(2)
s1.pop() = 1 → s2.push(1)
Now: s1 = [], s2 = [3, 2, 1]
Top of s2 = 1
→ Returns: 1
pop():
→ s2 not empty → pop from s2
s2.pop() = 1
Now: s1 = [], s2 = [3, 2]
→ Returns: 1
peek():
→ s2 not empty
Top of s2 = 2
→ Returns: 2
empty():
s1 = [], s2 = [3, 2] → Not empty
→ Returns: false
```

Final State:	s1 = []	, s2 = [3, 2]	(top to bottom)
---------------------	---------	---------------	-----------------

```
var MyQueue = function() {
  this.s1 = [];
  this.s2 = [];
};
MyQueue.prototype.push = function(x) {
  this.s1.push(x);
};
MyQueue.prototype.pop = function() {
  if(this.s2.length === 0) {
    while(this.s1.length) {
      this.s2.push(this.s1.pop());
    }
  }
  return this.s2.pop();
};
MyQueue.prototype.peek = function() {
  if(this.s2.length === 0) {
```

```

        while(this.s1.length) {
            this.s2.push(this.s1.pop());
        }
    }
    return this.s2[this.s2.length-1];
};

MyQueue.prototype.empty = function() {
    return this.s1.length === 0 && this.s2.length === 0;
};

```

Valid Parentheses

Problem Statement:

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

- Open brackets must be closed by the same type of brackets.
- Open brackets must be closed in the correct order.
- Every close bracket has a corresponding open bracket of the same type.

Example 1:

Input: *s* = "()"

Output: true

Example 2:

Input: *s* = "()[]{}"

Output: true

Example 3:

Input: *s* = "()["

Output: false

Example 4:

Input: s = "([])"

Output: true

Example 5:

Input: s = "([)]"

Output: false

Constraints:

- `1 <= s.length <= 104`
- `s` consists of parentheses only `'()[]{}'`.

Approach:

- Use a **stack** to track opening brackets.
- Create a **map** that stores matching pairs → `{ : }, [ : ], ( : )`.
- Traverse the string:
 - If it's an **opening bracket**, push it onto the stack.
 - If it's a **closing bracket**, pop from the stack and check if it matches the expected closing bracket. If not, **return false**.
- After processing, if the stack is empty → all brackets matched → **return true**. Otherwise, **return false**.

Time & Space Complexity:

Time Complexity: `O(n)`

Space Complexity: `O(n)`

Dry Run

Input: `s = "([{}])"`

Step 0: Start Function `isValid("([{}])")`
`stack = []`
`map = { "{" : "}", "[" : "]", "(" : ")" }`

Iteration (i = 0): `s[0] = "("`
- "(" is in map → push to stack
- `stack = ["("]`

Iteration (i = 1): `s[1] = "["`
- "[" is in map → push to stack
- `stack = ["(", "["]`

Iteration (i = 2): `s[2] = "{"`
- "{" is in map → push to stack
- `stack = ["(", "[", "{"]`

Iteration (i = 3): `s[3] = "}"`
- "}" is not in map → pop from stack
- `top = "{"`
- Check `s[3] == map[top] → "}" == map["{"] → "}" == "}"` ✓
- `stack = ["(", "["]`

Iteration (i = 4): `s[4] = "]"`
- "]" is not in map → pop from stack
- `top = "["`
- Check `s[4] == map[top] → "]" == map["["] → "]" == "]"` ✓
- `stack = ["("]`

Iteration (i = 5): `s[5] = ")"`
- ")" is not in map → pop from stack
- `top = "("`
- Check `s[5] == map[top] → ")" == map["("] → ")" == ")"` ✓
- `stack = []`

Loop ends (i == s.length)

Final Check:
stack.length === 0 → true
Return true

Output: true

```
var isValid = function(s){
  let stack = [];

  let map = {
    "{" : "}",
    "[" : "]",
    "(" : ")"
  }

  for(let i=0; i < s.length; i++){
    if(map[s[i]]) {
      stack.push(s[i]);
    }
    else {
      let top = stack.pop();
      if(!top || s[i] !== map[top]) {
        return false;
      }
    }
  }
  return stack.length === 0;
}
```

Min Stack

Problem Statement:

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.

Implement the `MinStack` class:

- `MinStack()` initializes the stack object.
- `void push(int val)` pushes the element `val` onto the stack.
- `void pop()` removes the element on the top of the stack.
- `int top()` gets the top element of the stack.
- `int getMin()` retrieves the minimum element in the stack.

You must implement a solution with `O(1)` time complexity for each function.

Examples:

Example 1:

Input:

["MinStack","push","push","push","getMin","pop","top","getMin"]
 [[],[-2],[0],[-3],[],[],[],[]]

Output: [null,null,null,null,-3,null,0,-2]

Explanation

```
MinStack minStack = new MinStack();
minStack.push(-2);
minStack.push(0);
minStack.push(-3);
minStack.getMin(); // return -3
minStack.pop();
minStack.top();    // return 0
minStack.getMin(); // return -2
```

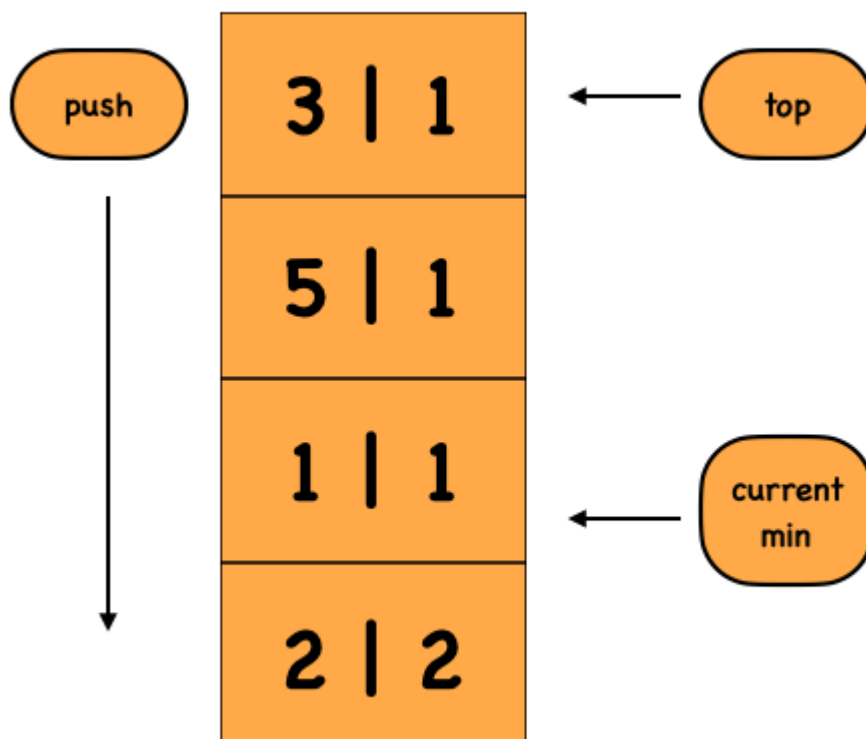
Constraints:

- $-2^{31} \leq \text{val} \leq 2^{31} - 1$
- Methods `pop`, `top` and `getMin` operations will always be called on **non-empty** stacks.
- At most $3 * 10^4$ calls will be made to `push`, `pop`, `top`, and `getMin`

Approach:

- Initialize stack, Use an array `s` to store pairs `[value, currentMin]`.
- `push(val)`
 - If stack is empty, push `[val, val]`.
 - Else, push `[val, min(val, top's min)]`.
- `pop()`: Remove the top element using `pop()`.
- `top()`: Return the top value: `s[s.length - 1][0]`.
- `getMin()`: Return the current minimum: `s[s.length - 1][1]`.

Visualisation:



Time Complexity:

- All operations: **$O(1)$**

Space Complexity:

- **Space Complexity = $O(n)$**

Dry Run:

Input:

Operations: push(3) push(5) push(2) push(1) getMin() pop()
getMin() top()

State Transitions:

```
After push(3): s = [[3, 3]]
After push(5):
min(5, 3) = 3 → push [5, 3]
s = [[3, 3], [5, 3]]
After push(2):
min(2, 3) = 2 → push [2, 2]
s = [[3, 3], [5, 3], [2, 2]]
After push(1):
min(1, 2) = 1 → push [1, 1]
s = [[3, 3], [5, 3], [2, 2], [1, 1]]
getMin():
Top element = [1, 1] → min = 1
pop():
Remove top element → [1, 1]
s = [[3, 3], [5, 3], [2, 2]]
getMin():
Top element = [2, 2] → min = 2
top():
Top element = [2, 2] → value = 2
```

Final State: `s = [[3, 3], [5, 3], [2, 2]]` (top to bottom: last element is top)

```

var MinStack = function() {
    this.s = [];
};
MinStack.prototype.push = function(val) {
    if(this.s.length === 0){
        this.s.push([val, val]);
    } else {
        let minVal = Math.min(val, this.s[this.s.length - 1][1]);
        this.s.push([val, minVal]);
    }
};
MinStack.prototype.pop = function() {
    this.s.pop();
};
MinStack.prototype.top = function() {
    return this.s[this.s.length-1][0];
};
MinStack.prototype.getMin = function() {
    return this.s[this.s.length-1][1];
};

```

Remove Outermost Parentheses

Problem Statement:

A valid parentheses string is either empty `""`, `"(" + A + ")"`, or `A + B`, where `A` and `B` are valid parentheses strings, and `+` represents string concatenation.

- For example, `""`, `"()"`, `"(())()"`, and `"(()(()))"` are all valid parentheses strings.

A valid parentheses string `s` is primitive if it is nonempty, and there does not exist a way to split it into `s = A + B`, with `A` and `B` nonempty valid parentheses strings.

Given a valid parentheses string `s`, consider its primitive decomposition: `s = P1 + P2 + ... + Pk`, where `Pi` are primitive valid parentheses strings.

Return `s` after removing the outermost parentheses of every primitive string in the primitive decomposition of `s`.

Examples:

Example 1:

Input:

`s = "(()())"`

Output: `"() ()"`

Explanation

The input string is `"(()())"`, with primitive decomposition `"(())" + "()"`. After removing outer parentheses of each part, this is `"()" + "()" = "() ()"`.

Example 2:

Input:

`s = "(()())()(()())"`

Output: `"() () () ()"`

Explanation

The input string is `"(()())()(()())"`, with primitive decomposition `"(())" + "()" + "(()())"`. After removing outer parentheses of each part, this is `"()" + "()" + "()()" = "() () ()"`.

Example 2:

Input:

`s = "()"`

Output: `" "`

Explanation

The input string is "()()", with primitive decomposition "()" + "()". After removing outer parentheses of each part, this is "" + "" = "".

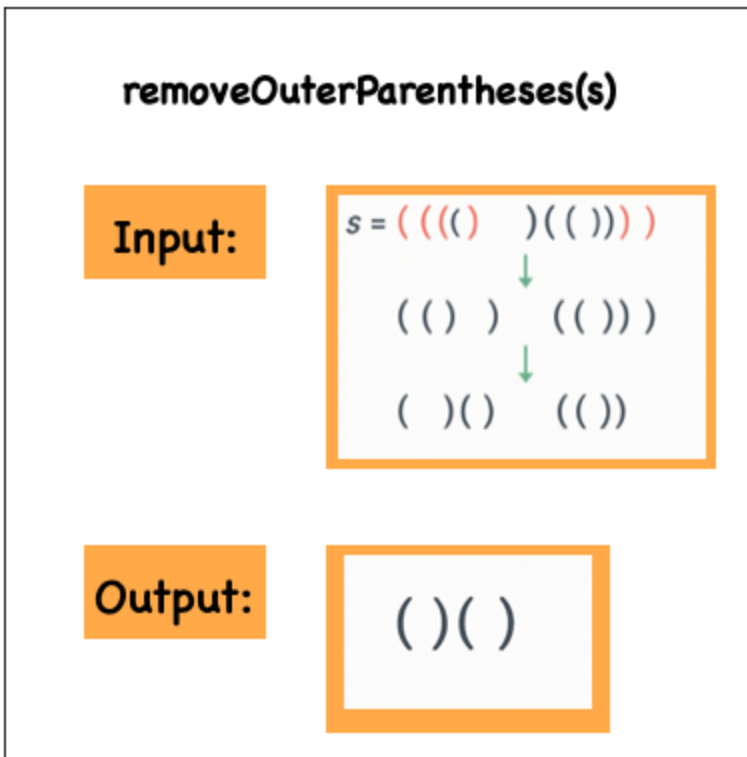
Constraints:

- `1 <= s.length <= 105`
- `s[i]` is either `'('` or `')'`.
- `s` is a valid parentheses string.

Approach:

- Initialize:
 - `stack` to track open parentheses.
 - `ans` to build the final string without outer parentheses.
- Loop through the string:
 - If character is `'('`
 - Push to `stack`.
 - Add to `ans` **only if** stack size > 1 (not outermost).
 - If character is `')'`
 - Add to `ans` **only if** stack size > 1 (not outermost).
 - Pop from `stack`.
- **Return** the final `ans` string.

Visualisation:



Time Complexity:

- Time Complexity = $O(n)$

Space Complexity:

- Space Complexity = $O(n)$

Dry Run

Input:

```
s = "((()())(())(()()))"
```

State Transitions:

```
Initialize: stack = [] ans = ""  
i = 0 → s[0] = "("  
→ stack.push("(") → stack = ["("]  
→ stack.length = 1 → Do NOT add to ans
```

```
i = 1 → s[1] = "("
→ stack.push("(") → stack = ["(", "("]
→ stack.length = 2 → ans += "(" → ans = "("

i = 2 → s[2] = ")"
→ stack.length = 2 → ans += ")" → ans = "()"
→ stack.pop() → stack = ["("]

i = 3 → s[3] = ")"
→ stack.length = 1 → Do NOT add to ans
→ stack.pop() → stack = []

i = 4 → s[4] = "("
→ stack.push("(") → stack = ["("]
→ stack.length = 1 → Do NOT add to ans

i = 5 → s[5] = "("
→ stack.push("(") → stack = ["(", "("]
→ stack.length = 2 → ans += "(" → ans = "()(("

i = 6 → s[6] = ")"
→ stack.length = 2 → ans += ")" → ans = "()()"
→ stack.pop() → stack = ["("]

i = 7 → s[7] = ")"
→ stack.length = 1 → Do NOT add to ans
→ stack.pop() → stack = []

i = 8 → s[8] = "("
→ stack.push("(") → stack = ["("]
→ stack.length = 1 → Do NOT add to ans

i = 9 → s[9] = "("
→ stack.push("(") → stack = ["(", "("]
→ stack.length = 2 → ans += "(" → ans = "()()(("

i = 10 → s[10] = ")"
→ stack.length = 2 → ans += ")" → ans = "()()()"
→ stack.pop() → stack = ["("]
```

```
i = 11 → s[11] = "("
→ stack.push("(") → stack = ["(", "("]
→ stack.length = 2 → ans += "(" → ans = "()()()("

i = 12 → s[12] = "("
→ stack.push("(") → stack = ["(", "(", "("]
→ stack.length = 3 → ans += "(" → ans = "()()()((("

i = 13 → s[13] = ")"
→ stack.length = 3 → ans += ")" → ans = "()()()((())"
→ stack.pop() → stack = ["(", "("]

i = 14 → s[14] = ")"
→ stack.length = 2 → ans += ")" → ans = "()()()(((())"
→ stack.pop() → stack = ["("]

i = 15 → s[15] = ")"
→ stack.length = 1 → Do NOT add to ans
→ stack.pop() → stack = []
```

Final Output: `() () () ((())`

Final State: `stack = [], ans = "() () () ((())"`

```
var removeOuterParentheses = function(s) {
  let stack = [];
  let ans = "";

  for(let i=0; i < s.length; i++) {
    if(s[i] === "(") {
      stack.push(i);
    } else {
      if(stack.length > 0) {
        ans += s[i];
        stack.pop();
      }
    }
  }
  return ans;
};
```

Remove Outermost Parentheses

Problem Statement:

A valid parentheses string is either empty $""$, $"(" + A + ")"$, or $A + B$, where A and B are valid parentheses strings, and $+$ represents string concatenation.

- For example, $""$, $"()"$, $"(())(())"$, and $"(()(()))"$ are all valid parentheses strings.

A valid parentheses string s is primitive if it is nonempty, and there does not exist a way to split it into $s = A + B$, with A and B nonempty valid parentheses strings.

Given a valid parentheses string s , consider its primitive decomposition: $s = P_1 + P_2 + \dots + P_k$, where P_i are primitive valid parentheses strings.

Return s after removing the outermost parentheses of every primitive string in the primitive decomposition of s .

Examples:

Example 1:

Input:

$s = "(())(())"$

Output: $"()()"$

Explanation

The input string is "`((()))()`", with primitive decomposition "`((()))`" + "`()`". After removing outer parentheses of each part, this is "`()`" + "`()`" = "`()()`".

Example 2:

Input:

`s = "((()))()((()))"`

Output: "`() () () () (())`"

Explanation

The input string is "`((()))()((()))`", with primitive decomposition "`((()))`" + "`()`" + "`((()))`". After removing outer parentheses of each part, this is "`()`" + "`()`" + "`((()))`" = "`()()((()))`".

Example 2:

Input:

`s = "()()`

Output: ""

Explanation

The input string is "`()()`", with primitive decomposition "`()`" + "`()`". After removing outer parentheses of each part, this is "" + "" = "".

Constraints:

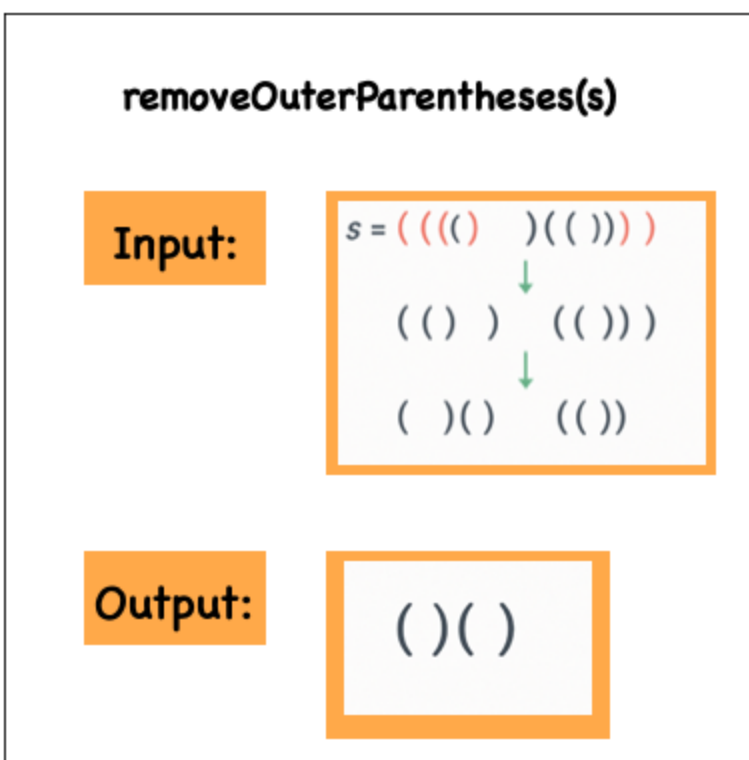
- `1 <= s.length <= 105`
- `s[i]` is either `'('` or `')'`.
- `s` is a valid parentheses string.

Second Approach: Without Stack

- Initialize: a counter `level` to `-1` and an empty string `ans`.
- Traverse** each character of the string `s`.

- If the character is '(',
 - Increment `level`.
 - If `level` is not 0, append '(' to `ans`.
- If the character is ')',
 - If `level` is not 0, append ')' to `ans`.
 - Then decrement `level`.
- **Return** the final `ans` string.

Visualisation:



Time Complexity:

- **Time Complexity = $O(n)$**

Space Complexity:

- **Space Complexity = $O(1)$**

Dry Run

Input:

```
s = "(()())(()(()))"
```

State Transitions:

Initialize: level = -1, ans = ""

i = 0 → s[0] = "("
→ ++level → level = 0
→ level is 0 → Do NOT add to ans

i = 1 → s[1] = "("
→ ++level → level = 1
→ level is non-zero → ans += "(" → ans = "("

i = 2 → s[2] = ")"
→ level is non-zero → ans += ")" → ans = "()"
→ --level → level = 0

i = 3 → s[3] = ")"
→ level is 0 → Do NOT add to ans
→ --level → level = -1

i = 4 → s[4] = "("
→ ++level → level = 0
→ level is 0 → Do NOT add to ans

i = 5 → s[5] = "("
→ ++level → level = 1
→ level is non-zero → ans += "(" → ans = "()(("

i = 6 → s[6] = ")"
→ level is non-zero → ans += ")" → ans = "(()())"
→ --level → level = 0

```
i = 7 → s[7] = ")"
→ level is 0 → Do NOT add to ans
→ --level → level = -1

i = 8 → s[8] = "("
→ ++level → level = 0
→ level is 0 → Do NOT add to ans

i = 9 → s[9] = "("
→ ++level → level = 1
→ level is non-zero → ans += "(" → ans = "()()("

i = 10 → s[10] = ")"
→ level is non-zero → ans += ")" → ans = "()()()"
→ --level → level = 0

i = 11 → s[11] = "("
→ ++level → level = 1
→ level is non-zero → ans += "(" → ans = "()()()("

i = 12 → s[12] = "("
→ ++level → level = 2
→ level is non-zero → ans += "(" → ans = "()()()((("

i = 13 → s[13] = ")"
→ level is non-zero → ans += ")" → ans = "()()()(((("
→ --level → level = 1

i = 14 → s[14] = ")"
→ level is non-zero → ans += ")" → ans = "()()()((((("
→ --level → level = 0

i = 15 → s[15] = ")"
→ level is 0 → Do NOT add to ans
→ --level → level = -1
```

Final Output: () () () (())

Final State: level = -1, ans = "()()()(())"

```
var removeOuterParentheses = function(s) {  
  let level = -1;  
  let ans = "";  
  for (let i = 0; i < s.length; i++) {  
    if (s[i] === "(") {  
      ++level;  
      ans += (level ? s[i] : "");  
    } else {  
      ans += (level ? s[i] : "");  
      --level;  
    }  
  }  
  return ans;  
};
```

Evaluate Reverse Polish Notation

Problem Statement:

You are given an array of strings `tokens` that represents an arithmetic expression in a Reverse Polish Notation.

Evaluate the expression. Return an integer that represents the value of the expression.

Note that:

- The valid operators are `'+'`, `'-'`, `'*'`, and `'/'`.
- Each operand may be an integer or another expression.
- The division between two integers always **truncates toward zero**.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a **32-bit** integer.

Examples:

Example 1:

Input:

tokens = ["2","1","+","3","*"]

Output: 9

Explanation

$$((2 + 1) * 3) = 9$$

Example 2:

Input:

tokens = ["4","13","5","/","+"]

Output: 6

Explanation

$$(4 + (13 / 5)) = 6$$

Example 3:

Input:

tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]

Output: 22

Explanation

$$\begin{aligned} & ((10 * (6 / ((9 + 3) * -11))) + 17) + 5 \\ & = ((10 * (6 / (12 * -11))) + 17) + 5 \\ & = ((10 * (6 / -132)) + 17) + 5 \\ & = ((10 * 0) + 17) + 5 \\ & = (0 + 17) + 5 \end{aligned}$$

```
= 17 + 5  
= 22
```

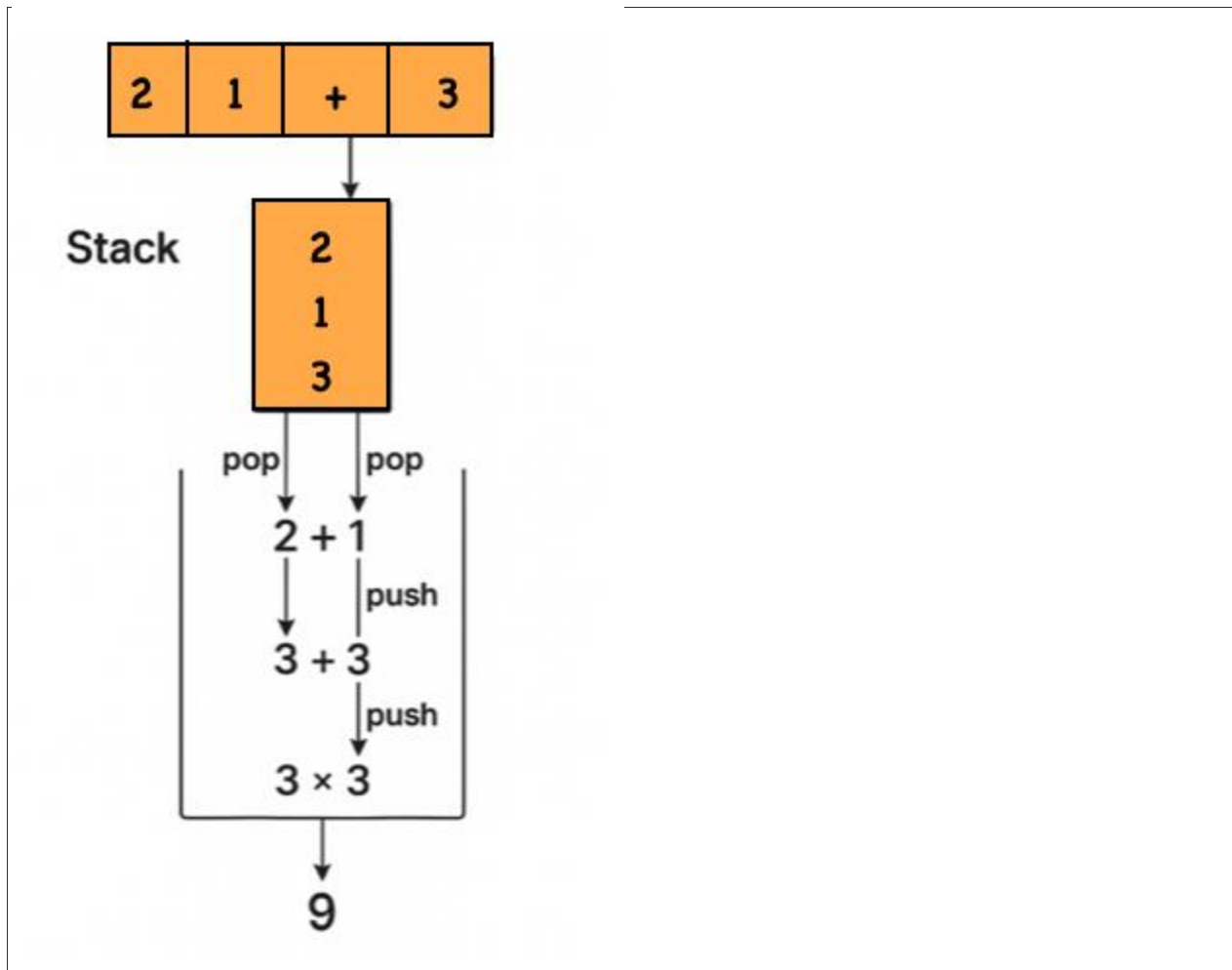
Constraints:

- `1 <= tokens.length <= 10`
- `tokens[i]` is either an operator: `"+"`, `"-"`, `"*"`, or `"/"`, or an integer in the range `[-200, 200]`.

Approach:

- **Initialize a stack** to store operands.
- **Define operator functions** using a map:
 - Each key `(+, -, *, /)` maps to a function that performs the respective operation on two numbers.
- **Loop through each token** in the input array:
 - If the token is an **operator**:
 - Pop the top two values from the stack.
 - Convert them to numbers and apply the operator function.
 - Push the result back to the stack
 - If the token is a **number**:
 - Push it directly to the stack.
 - **Return the final result** by popping the last element from the stack and converting it to a number.

Visualisation:



Time Complexity:

- **Time Complexity = $O(n)$**

Space Complexity:

- **Space Complexity = $O(n)$**

Dry Run

Input:

```
arr = ["2", "1", "+", "3", "*"]
```

State Transitions:

```
Initialize: stack = []  
i = 0 → arr[0] = "2"  
→ Not an operator → stack.push("2") → stack = ["2"]  
  
i = 1 → arr[1] = "1"  
→ Not an operator → stack.push("1") → stack = ["2", "1"]  
  
i = 2 → arr[2] = "+"  
→ Operator found  
→ a = stack.pop() = "1"  
→ b = stack.pop() = "2"  
→ ans = map["+"](1, 2) = 3  
→ stack.push(3) → stack = [3]  
  
i = 3 → arr[3] = "3"  
→ Not an operator → stack.push("3") → stack = [3, "3"]  
  
i = 4 → arr[4] = ""  
→ Operator found  
→ a = stack.pop() = "3"  
→ b = stack.pop() = 3  
→ ans = map[""](3, 3) = 9  
→ stack.push(9) → stack = [9]
```

Final Output: 9

Final State: stack = [] (after popping result)

```

var evalRPN = function(arr) {
    let stack = [];
    const map = {
        "+": (a,b) => (b+a),
        "*": (a,b) => (b*a),
        "-": (a,b) => (b-a),
        "/": (a,b) => Math.trunc(b/a),
    };
    for(let i=0; i < arr.length; i++){
        if(map[arr[i]]) {
            let a = stack.pop();
            let b = stack.pop();
            let ans = map[arr[i]](+a,+b);
            stack.push(ans);
        } else {
            stack.push(arr[i])
        }
    }
    return Number(stack.pop());
};

```

Next Greater Element

Problem Statement:

The **next greater element** of some element x in an array is the **first greater** element that is **to the right** of x in the same array.

You are given two **distinct 0-indexed** integer arrays `nums1` and `nums2`, where `nums1` is a subset of `nums2`.

For each `0 ≤ i < nums1.length`, find the index `j` such that `nums1[i] == nums2[j]` and determine the **next greater element** of `nums2[j]` in `nums2`. If there is no next greater element, then the answer for this query is `-1`.

Return an array `ans` of length `nums1.length` such that `ans[i]` is the **next greater element** as described above.

Examples:

Example 1:

Input:

nums1 = [4,1,2], nums2 = [1,3,4,2]

Output: [-1, 3, -1]

Explanation

The next greater element for each value of nums1 is as follows: - 4 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1. - 1 is underlined in nums2 = [1,3,4,2]. The next greater element is 3. - 2 is underlined in nums2 = [1,3,4,2]. There is no next greater element, so the answer is -1.

Example 2:

Input:

nums1 = [2,4], nums2 = [1,2,3,4]

Output: [3, -1]

Explanation

The next greater element for each value of nums1 is as follows: - 2 is underlined in nums2 = [1,2,3,4]. The next greater element is 3. - 4 is underlined in nums2 = [1,2,3,4]. There is no next greater element, so the answer is -1.

Constraints:

- `1 <= nums1.length <= nums2.length <= 1000`
- `0 <= nums1[i], nums2[i] <= 104`
- All integers in `nums1` and `nums2` are **unique**.

Approach:

- Initialize an empty map to store each element's next greater in `nums2`

- Use a stack to process `nums2` from right to left.
- For each element:
 - Pop smaller elements from the stack (they can't be NGE).
 - If stack has a greater element, that's the NGE.
 - If not, NGE is -1.
 - Push current element onto the stack.
- **Build the answer** by mapping elements from `nums1` using the precomputed values from the map.
- Return the result array.

Visualisation:

Time Complexity:

- **Time Complexity = $O(n + m)$**

Space Complexity:

- **Space Complexity = $O(n)$**

Dry Run

Input:

```
nums1 = [4, 1, 2]
nums2 = [1, 3, 4, 2]
```

State Transitions:

```
Initialize: stack = []
n = 4 → arr = [1, 3, 4, 2]
```

```
Push arr[3] = 2 onto stack
stack = [2]
ngeMap[2] = -1
```

```
i = 2 → arr[2] = 4
```

```
→ top = stack[stack.length-1] = 2
→ 4 > 2 → pop 2 → stack = []
→ stack empty → ngeMap[4] = -1
→ push 4 → stack = [4]
```

```
i = 1 → arr[1] = 3
→ top = 4
→ 3 < 4 → ngeMap[3] = 4
→ push 3 → stack = [4, 3]
```

```
i = 0 → arr[0] = 1
→ top = 3
→ 1 < 3 → ngeMap[1] = 3
→ push 1 → stack = [4, 3, 1]
```

```
Final ngeMap = {
  2: -1,
  4: -1,
  3: 4,
  1: 3
}
```

```
Build result from nums1 = [4, 1, 2]
→ ngeMap[4] = -1
→ ngeMap[1] = 3
→ ngeMap[2] = -1
→ ans = [-1, 3, -1]
```

Final Output: [-1, 3, -1]

Final State: stack = [4, 3, 1]

```
var nextGreaterElement = function(nums1, arr) {
  let ngeMap = {};
  let stack = [];
  let n = arr.length;
```

```

stack.push(arr[n-1]);
ngeMap[arr[n-1]] = -1;
for(let i=n-2; i>=0; i--){
    let top = stack[stack.length-1];
    if(arr[i] < top){
        ngeMap[arr[i]] = top;
    }
    else {
        while(stack.length) {
            if(stack[stack.length-1] < arr[i]){
                stack.pop();
            } else {
                ngeMap[arr[i]] = stack[stack.length-1];
                break;
            }
        }
        if(stack.length === 0){
            ngeMap[arr[i]] = -1;
        }
    }
    stack.push(arr[i]);
}
let ans = [];
for(let i=0; i < nums1.length; i++){
    ans.push(ngeMap[nums1[i]]);
}
return ans;
};

```

Daily Temperatures

Problem Statement:

Given an array of integers `temperatures` represents the daily temperatures, return an array `answer` such that `answer[i]` is the number of days you have to wait after the `i`th day to get a warmer temperature. If there is no future day for which this is possible, keep `answer[i] == 0` instead.

Examples:

Example 1:

Input:

temperatures = [73,74,75,71,69,72,76,73]

Output: [1,1,4,2,1,1,0,0]

Example 2:**Input:**

temperatures = [30,40,50,60]

Output: [1,1,1,0]

Example 2:**Input:**

temperatures = [30,60,90]

Output: [1,1,0]

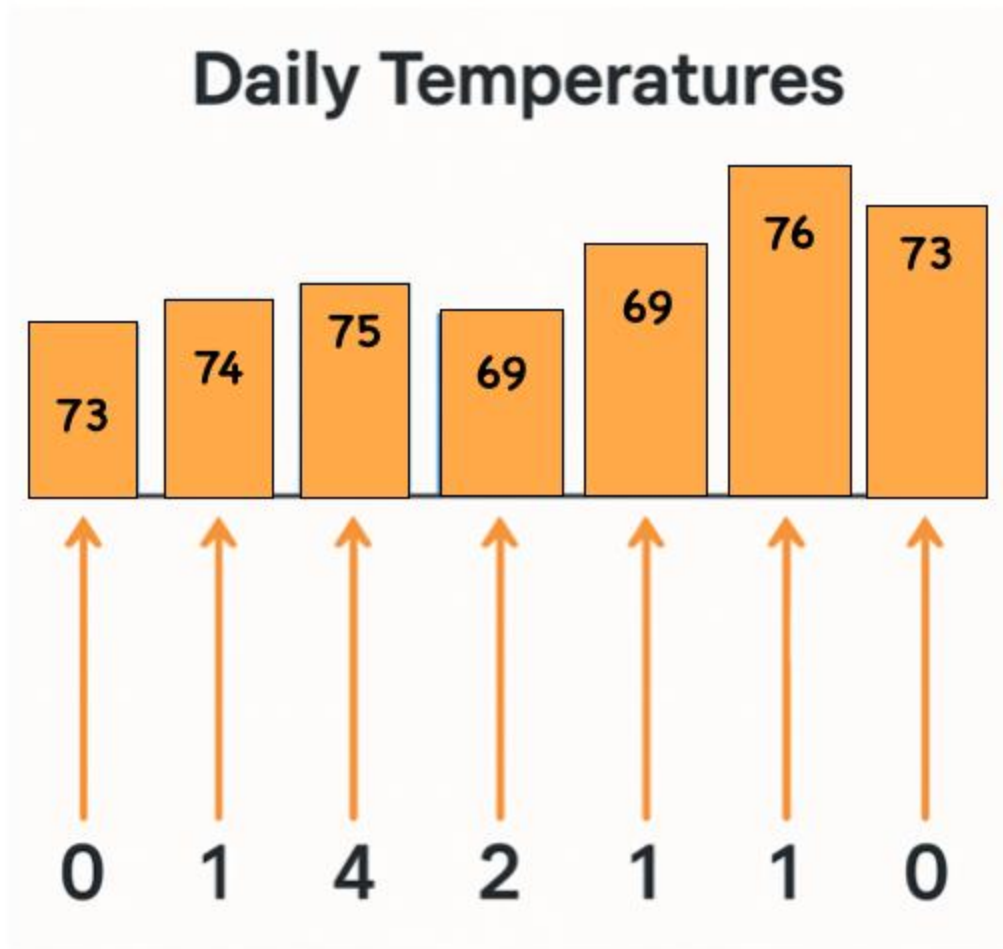
Constraints:

- `1 <= temperatures.length <= 10`
- `30 <= temperatures[i] <= 100`

Approach:

- Traverse from right to left (last to first day).
- Stack stores indices of days with temperatures in decreasing order.
- For each element:
 - Pop from stack until you find a warmer day.
 - If stack is not empty, the top is the next warmer day → store the difference.
 - Push current day index onto the stack.

Visualisation:



Time Complexity:

- Time Complexity = $O(n)$

Space Complexity:

- Space Complexity = $O(n)$

Dry Run

Input:

```
arr = [73, 74, 75, 71, 69, 72, 76, 73]
```

State Transitions:

Initialize: $n = 8$ $ans = [0, 0, 0, 0, 0, 0, 0, 0]$ $stack = []$
Push last index $\rightarrow stack = [7]$

$i = 6 \rightarrow arr[6] = 76$
 $\rightarrow top = 7 \rightarrow arr[6] \geq arr[7] (76 \geq 73) \rightarrow pop\ 7 \rightarrow stack = []$
 $\rightarrow stack\ empty \rightarrow no\ warmer\ day \rightarrow ans[6] = 0$
 $\rightarrow push\ 6 \rightarrow stack = [6]$

$i = 5 \rightarrow arr[5] = 72$
 $\rightarrow top = 6 \rightarrow arr[5] < arr[6] (72 < 76)$
 $\rightarrow ans[5] = 6 - 5 = 1$
 $\rightarrow push\ 5 \rightarrow stack = [6, 5]$

$i = 4 \rightarrow arr[4] = 69$
 $\rightarrow top = 5 \rightarrow arr[4] < arr[5] (69 < 72)$
 $\rightarrow ans[4] = 5 - 4 = 1$
 $\rightarrow push\ 4 \rightarrow stack = [6, 5, 4]$

$i = 3 \rightarrow arr[3] = 71$
 $\rightarrow top = 4 \rightarrow arr[3] \geq arr[4] (71 \geq 69) \rightarrow pop\ 4 \rightarrow stack = [6, 5]$
 $\rightarrow top = 5 \rightarrow arr[3] < arr[5] (71 < 72)$
 $\rightarrow ans[3] = 5 - 3 = 2$
 $\rightarrow push\ 3 \rightarrow stack = [6, 5, 3]$

$i = 2 \rightarrow arr[2] = 75$
 $\rightarrow top = 3 \rightarrow arr[2] \geq arr[3] (75 \geq 71) \rightarrow pop\ 3 \rightarrow stack = [6, 5]$
 $\rightarrow top = 5 \rightarrow arr[2] \geq arr[5] (75 \geq 72) \rightarrow pop\ 5 \rightarrow stack = [6]$
 $\rightarrow top = 6 \rightarrow arr[2] < arr[6] (75 < 76)$
 $\rightarrow ans[2] = 6 - 2 = 4$
 $\rightarrow push\ 2 \rightarrow stack = [6, 2]$

$i = 1 \rightarrow arr[1] = 74$
 $\rightarrow top = 2 \rightarrow arr[1] < arr[2] (74 < 75)$
 $\rightarrow ans[1] = 2 - 1 = 1$
 $\rightarrow push\ 1 \rightarrow stack = [6, 2, 1]$

$i = 0 \rightarrow arr[0] = 73$
 $\rightarrow top = 1 \rightarrow arr[0] < arr[1] (73 < 74)$

→ $\text{ans}[0] = 1 - 0 = 1$
→ push 0 → $\text{stack} = [6, 2, 1, 0]$

Final Output: `[1, 1, 4, 2, 1, 1, 0, 0]`

Final State: `stack = [6, 2, 1, 0]`

```
var dailyTemperatures = function(arr) {  
    let stack = [];  
    let n = arr.length;  
    let ans = Array(n).fill(0);  
  
    stack.push(n-1);  
    for(let i = n-2; i >= 0; i--){  
        while(stack.length) {  
            let top = stack[stack.length-1];  
            if(arr[i] >= arr[top]) {  
                stack.pop();  
            } else {  
                ans[i] = top - i;  
                break;  
            }  
        }  
        stack.push(i);  
    }  
    return ans;  
};
```

Next Greater Element II

Problem Statement:

Given a circular integer array `nums` (i.e., the next element of `nums[nums.length - 1]` is `nums[0]`), return the `next greater number` for every element in `nums`.

The **next greater number** of a number `x` is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return `-1` for this number.

Examples:

Example 1:

Input:

nums = [1,2,1]

Output: [2,-1,2]

Explanation: The first 1's next greater number is 2; The number 2 can't find next greater number. The second 1's next greater number needs to search circularly, which is also 2.

Example 2:

Input:

nums = [1,2,3,4,3]

Output: [2,3,4,-1,4]

Example 2:

Input:

temperatures = [30,40,50,60]

Output: [1,1,1,0]

Constraints:

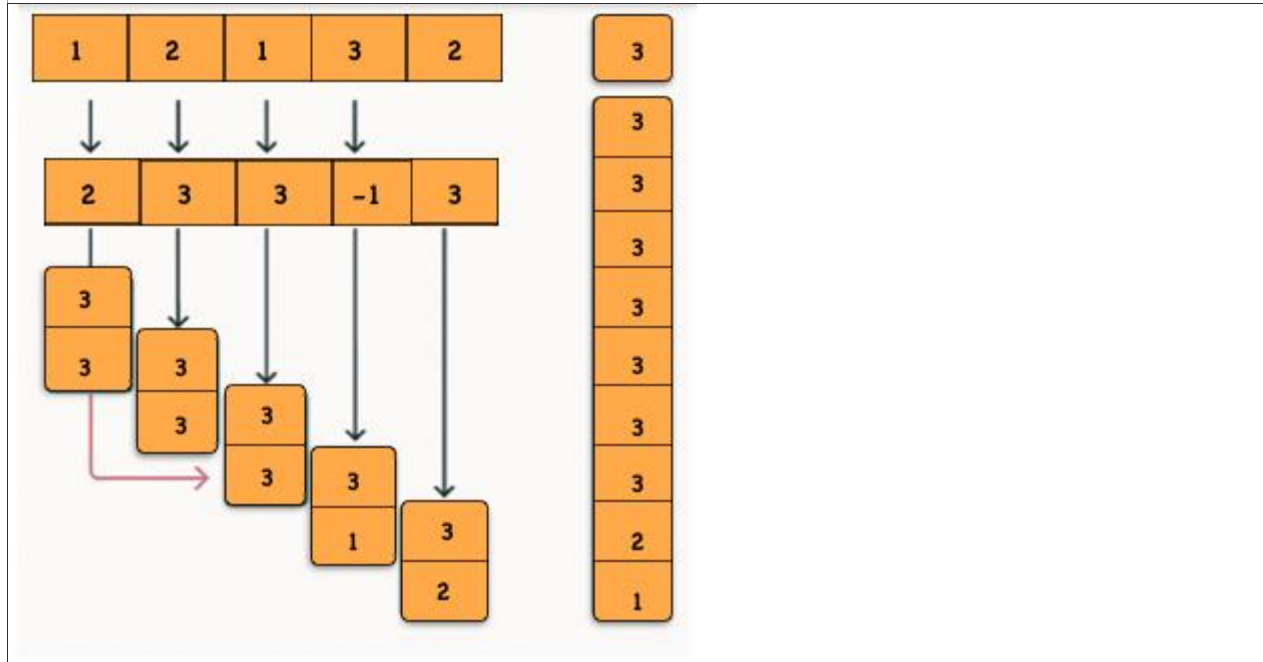
- `1 <= nums.length <= 104`
- `-109 <= temperatures[i] <= 109`

Approach:

- Initialize:
 - `n = arr.length`
 - `ans = Array(n).fill(-1)` → To store result.

- `stack = []` → To keep track of next greater candidates.
- **Push last element** into stack (start from end).
- **Traverse from $(2n - 2)$ to 0 :**
 - Use $i \% n$ to simulate circular behavior.
 - While stack is not empty:
 - If current element $<$ top of stack → that is the next greater, store it in `ans[i % n]` and break.
 - Else pop the stack (since it's not greater).
 - Push current element (`arr[i % n]`) onto the stack.
- **Return** `ans`

Visualisation:



Time Complexity:

- **Time Complexity = $O(n)$**

Space Complexity:

- **Space Complexity = $O(n)$**

Dry Run

Input:

```
arr = [73, 74, 75, 71, 69, 72, 76, 73]
```

State Transitions:

Initialize: $n = 8$ $ans = [-1, -1, -1, -1, -1, -1, -1, -1]$ $stack = []$

Push last element $\rightarrow stack = [73]$

$i = 14 \rightarrow arr[6] = 76$

$\rightarrow stack = [73]$

$\rightarrow arr[6] > top (76 > 73) \rightarrow ans[6] = -1$ (no change)

$\rightarrow push\ 76 \rightarrow stack = [76]$

$i = 13 \rightarrow arr[5] = 72$

$\rightarrow top = 76 \rightarrow arr[5] < top (72 < 76) \rightarrow ans[5] = 76$

$\rightarrow push\ 72 \rightarrow stack = [76, 72]$

$i = 12 \rightarrow arr[4] = 69$

$\rightarrow top = 72 \rightarrow arr[4] < top (69 < 72) \rightarrow ans[4] = 72$

$\rightarrow push\ 69 \rightarrow stack = [76, 72, 69]$

$i = 11 \rightarrow arr[3] = 71$

$\rightarrow top = 69 \rightarrow arr[3] > top \rightarrow pop\ 69$

$\rightarrow top = 72 \rightarrow arr[3] < top \rightarrow ans[3] = 72$

$\rightarrow push\ 71 \rightarrow stack = [76, 72, 71]$

$i = 10 \rightarrow arr[2] = 75$

$\rightarrow top = 71 \rightarrow arr[2] > top \rightarrow pop\ 71$

$\rightarrow top = 72 \rightarrow arr[2] > top \rightarrow pop\ 72$

$\rightarrow top = 76 \rightarrow arr[2] < top \rightarrow ans[2] = 76$

$\rightarrow push\ 75 \rightarrow stack = [76, 75]$

$i = 9 \rightarrow arr[1] = 74$

$\rightarrow top = 75 \rightarrow arr[1] < top \rightarrow ans[1] = 75$

$\rightarrow push\ 74 \rightarrow stack = [76, 75, 74]$

i = 8 → arr[0] = 73
→ top = 74 → arr[0] < top → ans[0] = 74
→ push 73 → stack = [76, 75, 74, 73]

i = 7 → arr[7] = 73
→ top = 73 → arr[7] == top → pop 73
→ top = 74 → arr[7] < top → ans[7] = 74
→ push 73 → stack = [76, 75, 74, 73]

i = 6 → arr[6] = 76
→ top = 73 → arr[6] > top → pop 73
→ top = 74 → arr[6] > top → pop 74
→ top = 75 → arr[6] > top → pop 75
→ top = 76 → arr[6] == top → pop 76
→ stack empty → ans[6] remains = -1
→ push 76 → stack = [76]

i = 5 → arr[5] = 72
→ top = 76 → arr[5] < top → ans[5] already set → push 72 →
stack = [76, 72]

i = 4 → arr[4] = 69
→ top = 72 → arr[4] < top → ans[4] already set → push 69 →
stack = [76, 72, 69]

i = 3 → arr[3] = 71
→ top = 69 → arr[3] > top → pop 69
→ top = 72 → arr[3] < top → ans[3] already set → push 71 →
stack = [76, 72, 71]

i = 2 → arr[2] = 75
→ top = 71 → arr[2] > top → pop 71
→ top = 72 → arr[2] > top → pop 72
→ top = 76 → arr[2] < top → ans[2] already set → push 75 →
stack = [76, 75]

i = 1 → arr[1] = 74
→ top = 75 → arr[1] < top → ans[1] already set → push 74 →
stack = [76, 75, 74]

$i = 0 \rightarrow \text{arr}[0] = 73$
 $\rightarrow \text{top} = 74 \rightarrow \text{arr}[0] < \text{top} \rightarrow \text{ans}[0]$ already set $\rightarrow$ push 73 $\rightarrow$
 $\text{stack} = [76, 75, 74, 73]$

Final Output: `[74, 75, 76, 72, 72, 76, -1, 74]`

Final State: `stack = [76, 75, 74, 73]`

```
var nextGreaterElements = function(arr) {
    let n = arr.length;
    let stack = [];
    let ans = Array(n).fill(-1);
    stack.push(arr[n-1]);
    for(let i=(2*n)-2; i >= 0; i--){
        while(stack.length){
            let top = stack[stack.length-1];
            if(arr[i % n] < top){
                ans[i % n] = top;
                break;
            } else{
                stack.pop();
            }
        }
        stack.push(arr[i % n]);
    }
    return ans.slice(0, n);
};
```

Rotting Oranges

Problem Statement:

You are given an `m x n` grid where each cell can have one of three values:

- `0` representing an empty cell,
- `1` representing a fresh orange, or
- `2` representing a rotten orange.

Every minute, any fresh orange that is **4-directionally adjacent** to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return `-1`.

Examples:

Example 1:

Input:

```
grid = [[2,1,1],[0,1,1],[1,0,1]]
```

Output: `-1`

Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens 4-directionally.

Example 2:

Input:

```
grid = [[0,2]]
```

Output: `0`

Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.

For more details, refer to LeetCode problem 994.

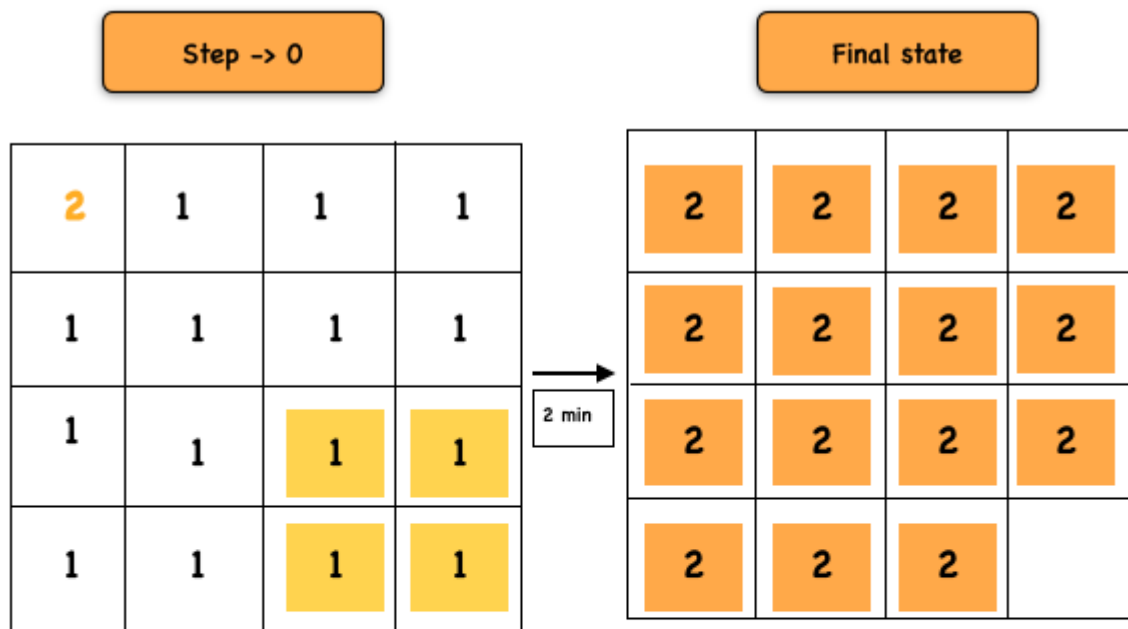
Constraints:

- `m == grid.length`
- `n == grid[i].length`
- `1 <= m, n <= 10`
- `grid[i][j]` is 0, 1, or 2.

Approach:

- **Identify rotten oranges** (2) and push their positions + time `([i, j, 0])` into a **queue**.
- **Use BFS** to spread rot in 4 directions (up, down, left, right).
 - For each direction: If neighbor is a fresh orange (1), rot it ($\rightarrow$ 2) and push it into the queue with time + 1.
- **Track time** with `maxMinutes`.
- After BFS, if any fresh orange (1) left, return `-1`.
- Else, return `maxMinutes`.

Visualisation:



Rotten orange



Fresh orange

2 min: The 2 minutes in the image represents the minimum time required to rot all fresh oranges in the grid.

Time Complexity:

- **Time Complexity = $O(m \times n)$**

Space Complexity:

- **Space Complexity = $O(m \times n)$**

Dry Run

Input:

```
grid = [ [2,1,1], [1,1,0], [0,1,1] ]
```

State Transitions:

```
Initialize: m = 3, n = 3 queue = []  
→ Scanning grid:  
Found rotten orange at (0,0) → queue = [[0,0,0]]
```

```
Start BFS:  
queue = [[0,0,0]]  
maxMinutes = 0
```

```
Pop [0,0,0]  
→ (0,1) is fresh → rot it → queue.push([0,1,1])  
→ (1,0) is fresh → rot it → queue.push([1,0,1])  
maxMinutes = max(0, 0) = 0
```

```
Pop [0,1,1]  
→ (0,2) is fresh → rot it → queue.push([0,2,2])  
→ (1,1) is fresh → rot it → queue.push([1,1,2])  
maxMinutes = max(1, 0) = 1
```

```
Pop [1,0,1]  
→ (2,0) is 0 (empty) → skip  
maxMinutes = max(1, 1) = 1
```

```
Pop [0,2,2]  
→ (1,2) is 0 → skip
```

maxMinutes = max(2, 1) = 2

Pop [1,1,2]

→ (2,1) is fresh → rot it → queue.push([2,1,3])

maxMinutes = max(2, 2) = 2

Pop [2,1,3]

→ (2,2) is fresh → rot it → queue.push([2,2,4])

maxMinutes = max(3, 2) = 3

Pop [2,2,4]

→ all neighbors are either 0 or already rotten

maxMinutes = max(4, 3) = 4

→ BFS finished, check for any remaining fresh oranges:

→ All are rotten or empty

Final Output: 4

Final State: grid = [[2,2,2], [2,2,0], [0,2,2]]

```
var orangesRotting = function(grid) {
    let m = grid.length;
    let n = grid[0].length;
    let queue = [];
    for (let i = 0; i < m; i++) {
        for (let j = 0; j < n; j++) {
            if (grid[i][j] === 2) {
                queue.push([i, j, 0]);
            }
        }
    }
    let maxMinutes = 0;
    while (queue.length) {
        let [x, y, level] = queue.shift();

        if (x > 0 && grid[x - 1][y] === 1) {
            grid[x - 1][y] = 2;
            queue.push([x - 1, y, level + 1]);
        }
    }
}
```

```
    if (x < m - 1 && grid[x + 1][y] === 1) {
        grid[x + 1][y] = 2;
        queue.push([x + 1, y, level + 1]);
    }
    if (y < n - 1 && grid[x][y + 1] === 1) {
        grid[x][y + 1] = 2;
        queue.push([x, y + 1, level + 1]);
    }
    if (y > 0 && grid[x][y - 1] === 1) {
        grid[x][y - 1] = 2;
        queue.push([x, y - 1, level + 1]);
    }
    maxMinutes = Math.max(level, maxMinutes);
}
for (let i = 0; i < m; i++) {
    for (let j = 0; j < n; j++) {
        if (grid[i][j] === 1) {
            return -1;
        }
    }
}
return maxMinutes;
};
```